



UNIVERSIDAD PERUANA UNIÓN

Escuela Profesional de Ingeniería de Sistemas

Informe de evidencia de aprendizaje

S05 Consultas Empresariales Reportes REST y CORS

SITRA-ORO

Sistema de Información, Trazabilidad y Liquidación en Acopio de Oro

1. Datos del estudiante

Nombre	Faijo Calisaya Helio Paul
Equipo	Equipo 05
Sesión	S05 - Consultas Empresariales, Reportes REST y CORS
Rol o aporte	Implementación y documentación de filtros, ordenamiento, proyección DTO, reporte agregado y CORS.
GitHub	https://github.com/helionelio900-hub/SysProyec_Acopus

2. Propósito

Esta actividad demuestra la implementación individual de consultas empresariales en el dominio de liquidaciones mayoristas de SITRA-ORO. La solución combina filtros opcionales, ordenamiento seguro, una proyección resumida, agregados que responden cero cuando no existen datos y una política CORS configurable.

En todas las capturas debe verse la ventana completa, el reloj con fecha y hora y el usuario o perfil. Las fechas deben ser coherentes con el historial de commits del repositorio.

3. Evidencia técnica

3.1 Filtros combinados y ordenamiento

GET /api/v1/mayorista/liquidaciones recibe los filtros opcionales estado, desde y hasta. Los parámetros ordenarPor y direccion permiten cambiar el orden, y el servicio valida que solo se soliciten campos permitidos y las direcciones ASC o DESC.

```
SELECT l FROM LiquidacionG1 l
WHERE (:estado IS NULL OR l.estado = :estado)
      AND (:desde IS NULL OR l.fechaLiquidacion >= :desde)
      AND (:hasta IS NULL OR l.fechaLiquidacion <= :hasta)
```

Captura 1. Filtro por estado

Ejecutar estado=REGISTRADA y dejar visibles la URL, la respuesta, el reloj y el perfil.

The screenshot shows a REST client interface with the following details:

- Method:** GET
- URL:** /api/v1/mayorista/liquidaciones
- Parameters:**
 - estado: REGISTRADA (dropdown)
 - desde: (empty text input)
 - hasta: (empty text input)
 - ordenarPor: fechaLiquidacion (dropdown)
 - direccion: DESC (dropdown)
- Buttons:** Execute, Clear
- Responses:**
 - curl:**

```
curl -X 'GET' \
  http://127.0.0.1:8081/api/v1/mayorista/liquidaciones?estado=REGISTRADA&ordenarPor=fechaLiquidacion&direccion=DESC' \
  -H 'accept: */*'
```
 - Request URL:** http://127.0.0.1:8081/api/v1/mayorista/liquidaciones?estado=REGISTRADA&ordenarPor=fechaLiquidacion&direccion=DESC
 - Server response:**
 - Code:** 200
 - Details:** Response body
 - Response body:**

```
{
  "idLiquidacionG1": 25,
  "nombreLiquidadorG1": "Acoplador Central Juilaca",
  "estado": "REGISTRADA",
  "pesoTotalUndeidadG1": 0.0,
  "cotizacionUndeidad": 1050,
  "tipoCombustible": 0.75,
  "totalPagadoUndeidad": 26250.0,
  "fechaLiquidacion": "2026-09-08T20:00:45.472513",
  "detalle": [
    {
      "idDetalleLiquidacion": 25,
      "tipoUnde": "RECIBO",
      "pesoUndeidad": 0.0
    }
  ]
}
```

Explicación. La respuesta contiene solo liquidaciones registradas y comprueba que el filtro por estado funciona de manera independiente.

Captura 2. Filtro por rango de fechas

Ejecutar con desde y hasta, usando un intervalo que contenga datos.

Parameters

Name	Description
estado	string (query)
desde	string (date-time) (query)
hasta	string (date-time) (query)
ordenarPor	string (query)
direccion	string (query)

Responses

curl -X 'GET' \

```
http://127.0.0.1:8081/api/v1/mayorista/liquidaciones?desde=2026-09-01T00:00:00&hasta=2026-09-10T23:59:59&ordenarPor=fechaLiquidacion&direccion=DESC \
```

Request URL

http://127.0.0.1:8081/api/v1/mayorista/liquidaciones?desde=2026-09-01T00:00:00&hasta=2026-09-10T23:59:59&ordenarPor=fechaLiquidacion&direccion=DESC

Server response

Code Details

200

Response body

```
{  "idLiquidacion": 25,  "nombreLiquidador": "Acopiador Central Juilaca",  "estado": "REGISTRADA",  "pesoTotalFundido": 0,  "confirmacionOrtodoxa": 100,  "timeCombinacion": 3.12,  "totalFogonados": 2026.0,  "fechaLiquidacion": "2026-09-08T20:00:45.472517",  "detalles": [    {      "idDetalleLiquidacion": 25,      "tipo": "REG",      "pesoTotalFundido": 0    }  ]}
```

Explicación. La consulta limita los registros por fechaLiquidacion. La evidencia debe mostrar resultados dentro del intervalo solicitado.

Captura 3. Filtros combinados

Ejecutar estado=REGISTRADA junto con desde y hasta.

Parameters

Name	Description
estado	string (query)
desde	string (date-time) (query)
hasta	string (date-time) (query)
ordenarPor	string (query)
direccion	string (query)

Responses

curl -X 'GET' \

```
http://127.0.0.1:8081/api/v1/mayorista/liquidaciones?estado=REGISTRADA&desde=2026-09-01T00:00:00&hasta=2026-09-10T23:59:59&ordenarPor=fechaLiquidacion&direccion=DESC \
```

Request URL

http://127.0.0.1:8081/api/v1/mayorista/liquidaciones?estado=REGISTRADA&desde=2026-09-01T00:00:00&hasta=2026-09-10T23:59:59&ordenarPor=fechaLiquidacion&direccion=DESC

Server response

Code Details

200

Response body

```
{  "idLiquidacion": 25,  "nombreLiquidador": "Acopiador Central Juilaca",  "estado": "REGISTRADA",  "pesoTotalFundido": 0,  "confirmacionOrtodoxa": 100,  "timeCombinacion": 3.12,  "totalFogonados": 2026.0,  "fechaLiquidacion": "2026-09-08T20:00:45.472517",  "detalles": [    {      "idDetalleLiquidacion": 25,      "tipo": "REG",      "pesoTotalFundido": 0    }  ]}
```

Explicación. La misma consulta aplica simultáneamente el estado y el intervalo de fechas; por eso los filtros son combinables.

Captura 4. Cambio de orden

Repetir una consulta cambiando direccion de DESC a ASC sobre fechaLiquidacion.

The screenshot shows a REST client interface with the following fields and values:

- `string($date-time) (query)`: `hasta` field with value `2026-09-10T23:59:59`
- `string($date-time) (query)`: `ordenarPor` field with value `fechaLiquidacion`
- `string (query)`: `direccion` field with value `ASC`

Buttons: `Execute` (highlighted in blue) and `Clear`.

Responses

Curl

```
curl -X 'GET' \
'http://127.0.0.1:8081/api/v1/mayorista/liquidaciones?estado=REGISTRADA&desde=2026-09-01T00%3A00%3A00&hasta=2026-09-10T23%3A59%3A59&ordenarPor=fechaLiquidacion&direccion=ASC' \
-H 'accept: */*'
```

Request URL

```
http://127.0.0.1:8081/api/v1/mayorista/liquidaciones?estado=REGISTRADA&desde=2026-09-01T00%3A00%3A00&hasta=2026-09-10T23%3A59%3A59&ordenarPor=fechaLiquidacion&direccion=ASC
```

Server response

Code **Details**

Explicación. El cambio visible en la secuencia de resultados confirma que el ordenamiento es configurable.

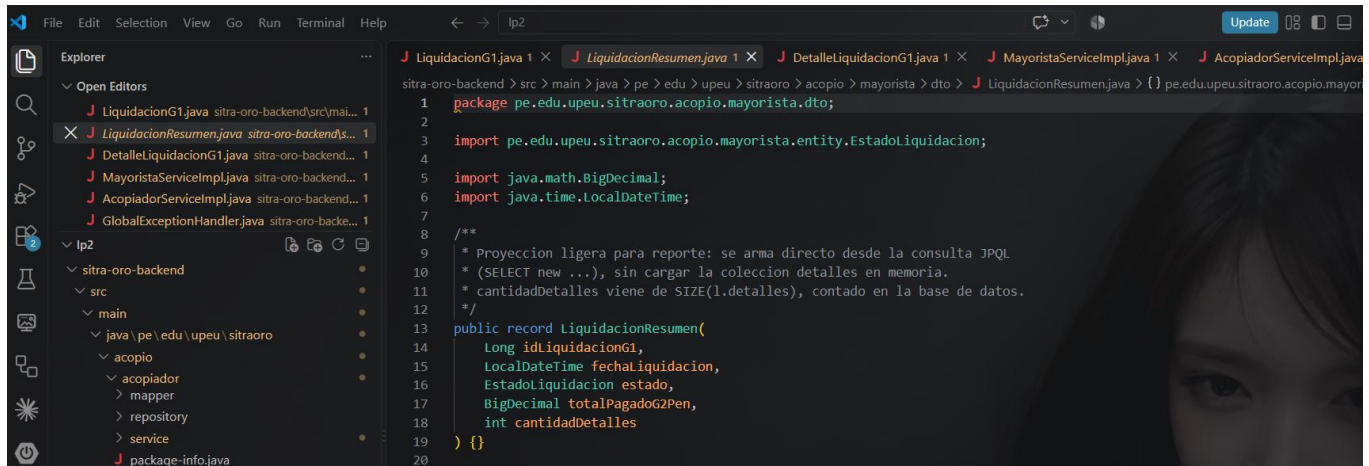
3.2 Proyección de resumen

LiquidacionResumen es un DTO distinto de la entidad completa. Transporta el identificador, fecha, estado, total pagado y cantidad de detalles para responder de forma ligera a un reporte.

```
public record LiquidacionResumen(  
    Long idLiquidacionG1, LocalDateTime fechaLiquidacion,  
    EstadoLiquidacion estado, BigDecimal totalPagadoG2Pen,  
    int cantidadDetalles  
) {}  
  
SELECT new ...LiquidacionResumen(  
    l.idLiquidacionG1, l.fechaLiquidacion, l.estado,  
    l.totalPagadoG2Pen, SIZE(l.detalles))
```

Captura 5. DTO LiquidacionResumen

Abrir *LiquidacionResumen.java* en VS Code y mostrar el record completo.

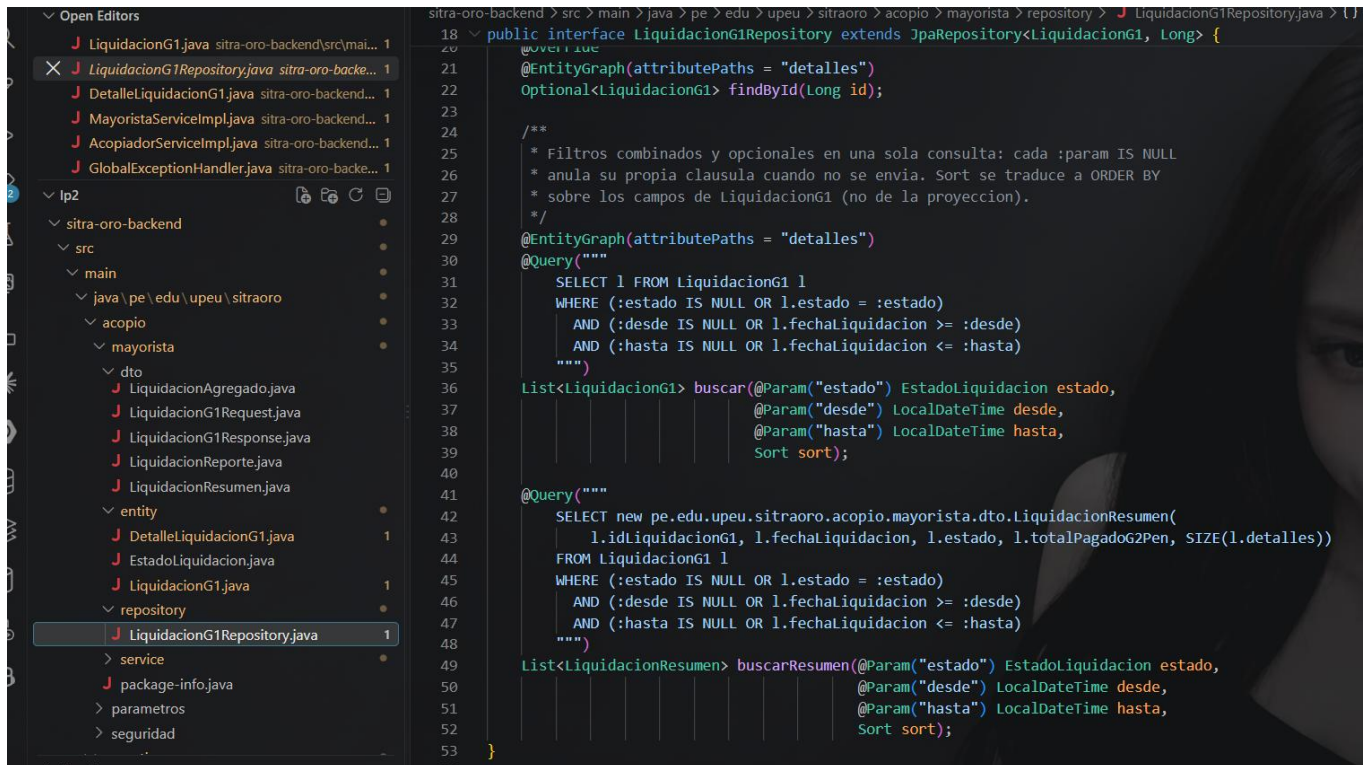


```
1 package pe.edu.upeu.sitraoro.acopio.mayorista.dto;
2
3 import pe.edu.upeu.sitraoro.acopio.mayorista.entity.EstadoLiquidacion;
4
5 import java.math.BigDecimal;
6 import java.time.LocalDateTime;
7
8 /**
9  * Proyeccion ligera para reporte: se arma directo desde la consulta JPQL
10  * (SELECT new ...), sin cargar la coleccion detalles en memoria.
11  * cantidadDetalles viene de SIZE(l.detalles), contado en la base de datos.
12  */
13 public record LiquidacionResumen(
14     Long idLiquidacionG1,
15     LocalDateTime fechaLiquidacion,
16     EstadoLiquidacion estado,
17     BigDecimal totalPagadoG2Pen,
18     int cantidadDetalles
19 ) {}
20
```

Explicación. El DTO expone únicamente los datos necesarios para el reporte y evita devolver la entidad completa.

Captura 6. Expresión de constructor JPQL

Abrir *LiquidacionG1Repository.java* y mostrar *SELECT new* junto con *buscarResumen(...)*.



```
18 public interface LiquidacionG1Repository extends JpaRepository<LiquidacionG1, Long> {
19     @Override
20     @EntityGraph(attributePaths = "detalles")
21     Optional<LiquidacionG1> findById(Long id);
22
23
24
25     /**
26      * Filtros combinados y opcionales en una sola consulta: cada :param IS NULL
27      * anula su propia clausula cuando no se envia. Sort se traduce a ORDER BY
28      * sobre los campos de LiquidacionG1 (no de la proyeccion).
29      */
30     @EntityGraph(attributePaths = "detalles")
31     @Query("""
32         SELECT l FROM LiquidacionG1 l
33         WHERE (:estado IS NULL OR l.estado = :estado)
34         AND (:desde IS NULL OR l.fechaLiquidacion >= :desde)
35         AND (:hasta IS NULL OR l.fechaLiquidacion <= :hasta)
36         """)
37     List<LiquidacionG1> buscar(@Param("estado") EstadoLiquidacion estado,
38                               @Param("desde") LocalDateTime desde,
39                               @Param("hasta") LocalDateTime hasta,
40                               Sort sort);
41
42     @Query("""
43         SELECT new pe.edu.upeu.sitraoro.acopio.mayorista.dto.LiquidacionResumen(
44             l.idLiquidacionG1, l.fechaLiquidacion, l.estado, l.totalPagadoG2Pen, SIZE(l.detalles))
45         FROM LiquidacionG1 l
46         WHERE (:estado IS NULL OR l.estado = :estado)
47         AND (:desde IS NULL OR l.fechaLiquidacion >= :desde)
48         AND (:hasta IS NULL OR l.fechaLiquidacion <= :hasta)
49         """)
50     List<LiquidacionResumen> buscarResumen(@Param("estado") EstadoLiquidacion estado,
51                                             @Param("desde") LocalDateTime desde,
52                                             @Param("hasta") LocalDateTime hasta,
53                                             Sort sort);
54 }
```

Explicación. La consulta construye *LiquidacionResumen* desde JPQL y obtiene *SIZE(l.detalles)* sin incluir las líneas completas.

3.3 Reporte agregado

GET /api/v1/mayorista/liquidaciones/resumen devuelve totalLiquidaciones, montoTotal y ticketPromedio junto con la lista resumida. El cálculo usa BigDecimal y cubre explícitamente el caso vacío con cero, no null.

```
long total = liquidaciones.size();
BigDecimal montoTotal = liquidaciones.stream()
    .map(LiquidacionResumen::totalPagadoG2Pen)
    .reduce(BigDecimal.ZERO, BigDecimal::add);
BigDecimal ticketPromedio = total == 0 ? BigDecimal.ZERO
    : montoTotal.divide(BigDecimal.valueOf(total), 2, RoundingMode.HALF_UP);
```

Captura 7. Reporte con resultados

Ejecutar /liquidaciones/resumen en un intervalo con registros.

GET

/api/v1/mayorista/liquidaciones/resumen

Reporte de liquidaciones: agregados (conteo, monto total, ticket promedio) y detalle resumido

Cancel

Parameters

Name	Description
estado	--
string (query)	
desde	2026-09-01T00:00:00
string(\$date-time) (query)	
hasta	2026-09-10T23:59:59
string(\$date-time) (query)	

ExecuteClear

Responses

Curl

curl -X 'GET' \n'http://127.0.0.1:8081/api/v1/mayorista/liquidaciones/resumen?desde=2026-09-01T00%3A00%3A00&hasta=2026-09-10T23%3A59%3A59' \n-H 'accept: */*'

Request URL

http://127.0.0.1:8081/api/v1/mayorista/liquidaciones/resumen?desde=2026-09-01T00%3A00%3A00&hasta=2026-09-10T23%3A59%3A59

Server response

Code	Details
200	Response body <pre>{ "agregado": { "totalLiquidaciones": 4, "montoTotal": 41313.0, "ticketPromedio": 20329.45 }, "liquidaciones": [{ "idLiquidacionG1": 25, "fechaLiquidacion": "2026-09-08T20:00:45.472517", "estado": "REGISTRADA", "totalPagadoG2Pen": 26039.4, "cantidadDetalles": 2 }, { "idLiquidacionG1": 24, "fechaLiquidacion": "2026-09-08T19:57:29.805142", "estado": "REGISTRADA", "totalPagadoG2Pen": 26039.4, "cantidadDetalles": 2 }, { "idLiquidacionG1": 23, "fechaLiquidacion": "2026-09-08T19:49:50.295409", "estado": "REGISTRADA", "totalPagadoG2Pen": 26039.4, "cantidadDetalles": 2 }] }</pre>

Explicación. El agregado muestra conteo, monto total y ticket promedio calculados a partir de las liquidaciones filtradas.

Captura 8. Reporte sin resultados

Ejecutar `/liquidaciones/resumen` con un rango futuro o sin coincidencias.

GET `/api/v1/mayorista/liquidaciones/resumen` Reporte de liquidaciones: agregados (conteo, monto total, ticket promedio) y detalle resumido

Parameters

Name	Description
estado	--
desde	2099-01-01T00:00:00
hasta	2099-12-31T23:59:59

Execute **Clear**

Responses

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8081/api/v1/mayorista/liquidaciones/resumen?desde=2099-01-01T00:00:00&hasta=2099-12-31T23:59:59' \
  -H 'accept: */*'
```

Request URL

```
http://127.0.0.1:8081/api/v1/mayorista/liquidaciones/resumen?desde=2099-01-01T00:00:00&hasta=2099-12-31T23:59:59
```

Server response

Code	Details
200	Response body <pre>{ "agregado": { "totalLiquidaciones": 0, "montoTotal": 0, "ticketPromedio": 0 }, "liquidaciones": [] }</pre> Response headers <pre>connection: keep-alive content-type: application/json date: Sun, 13 Sep 2026 14:23:53 GMT keep-alive: timeout=60 transfer-encoding: chunked vary: Origin,Access-Control-Request-Method,Access-Control-Request-Headers x-trace-id: 987b6280-90a0-4170-9ba5-34eae433782b</pre>

Download

Explicación. La respuesta debe mostrar `totalLiquidaciones`, `montoTotal` y `ticketPromedio` con valor 0, nunca null.

3.4 Configuración y prueba de CORS

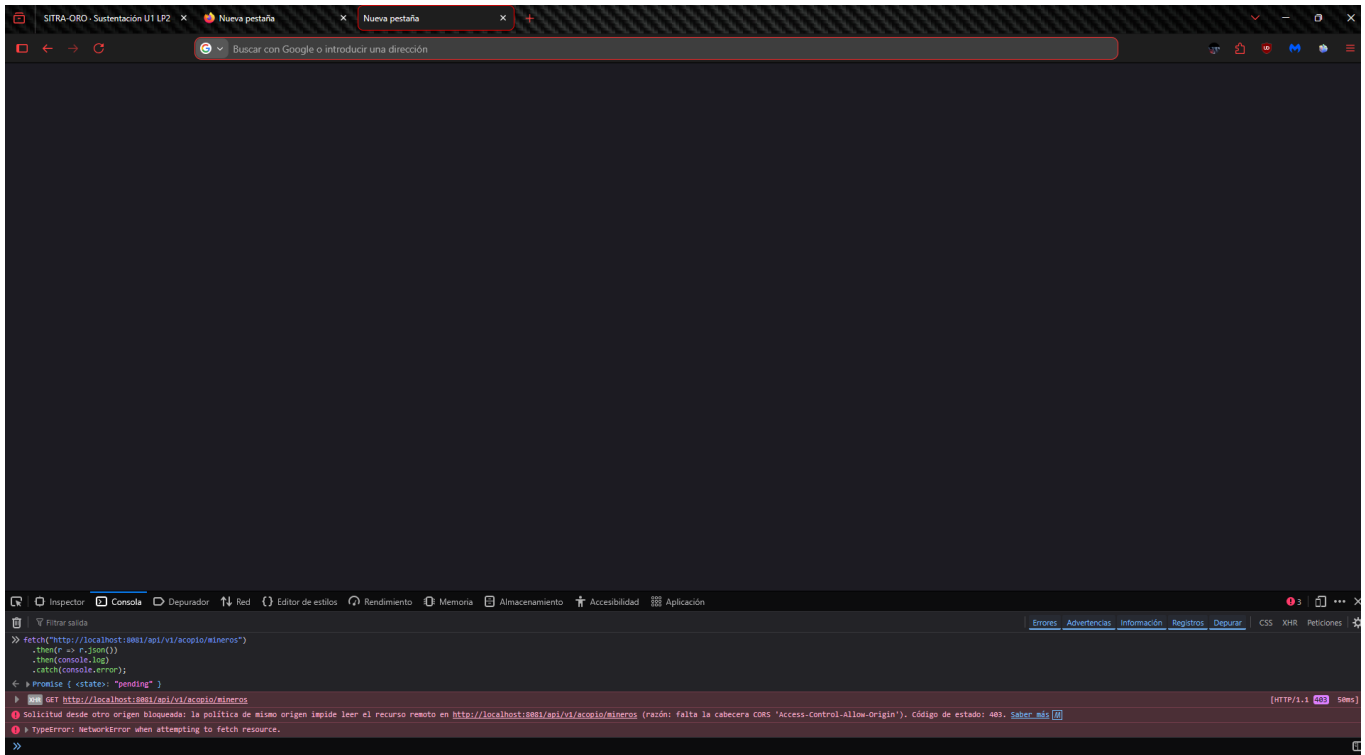
CorsConfig aplica la política a `/api/**` y lee el origen permitido desde `sitraoro.cors.allowed-origin`. En desarrollo se permite `http://localhost:4200`. La prueba debe realizarse desde JavaScript en un navegador real, no desde Postman ni consola.

```
@Value("${sitraoro.cors.allowed-origin:http://localhost:4200}")
private String allowedOrigin;

registry.addMapping("/api/**")
    .allowedOrigins(allowedOrigin)
    .allowedMethods("GET", "POST", "PUT", "PATCH", "DELETE")
    .allowedHeaders("*");
```

Captura 9. Origen no permitido bloqueado

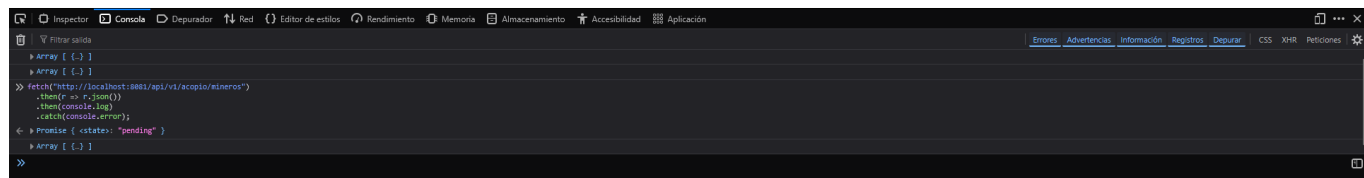
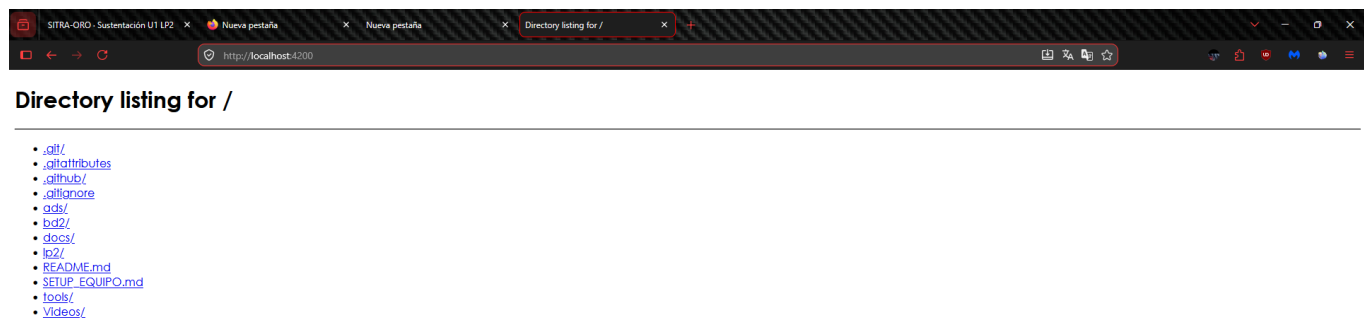
Desde un origen distinto a `http://localhost:4200`, ejecutar `fetch` y mostrar el error CORS en DevTools.



Explicación. El navegador bloquea el acceso de JavaScript a la respuesta cuando el Origen no coincide con el autorizado por el backend.

Captura 10. Origen configurado permitido

Desde `http://localhost:4200`, repetir `fetch` y mostrar una respuesta exitosa.



Explicación. Cuando el Origen coincide con la configuración, el navegador permite que JavaScript lea la respuesta de la API.

4. Hallazgo técnico

Ordenamiento dinámico sin validación

ordenarPor llega como texto por la URL. Si se usara sin validar, una propiedad inexistente provocaría un error en la consulta. La solución mantiene CAMPOS_ORDENABLES y acepta solamente ASC o DESC, con lo que la API rechaza pedidos inválidos de forma controlada.

5. Reflexión técnica breve

CORS no protege la API frente a curl o Postman porque la regla la hace cumplir el navegador, no el servidor como autenticación. Un cliente HTTP externo puede enviar peticiones directas. CORS protege al usuario web: impide que JavaScript de un sitio no autorizado lea respuestas de otro origen utilizando la sesión del usuario. Por eso no reemplaza autenticación, autorización ni validación de permisos en el backend.

6. Conclusiones

- Los filtros opcionales permiten resolver varias combinaciones con una única consulta.
- La proyección reduce los datos transportados hacia el cliente.
- Los agregados vacíos devuelven cero y evitan valores null en el frontend.
- CORS debe configurarse por ambiente y comprobarse en un navegador real.

Anexo Feedback de la sesión S05

1. ¿Cuál es el aprendizaje más importante?

Aprendí a crear consultas con filtros opcionales, ordenamiento y una proyección de reporte más ligera que la entidad completa.

2. ¿Qué punto fue más confuso?

Al inicio, comprender por qué AVG en JPQL devuelve Double aunque los montos del dominio sean BigDecimal.

3. ¿Qué pregunta queda para la siguiente clase?

¿Cuál es la mejor forma de paginar y exportar un reporte grande sin cargar todos los resultados en memoria?

4. Nivel de comprensión

[X] ¡Entendido! - Lo domino y podría explicarlo.

5. ¿Cómo puedo ayudarte a comprender mejor?

Con un ejemplo que compare proyecciones por constructor, interfaces de proyección y consultas nativas.

6. Participación y esfuerzo

[X] Muy Comprometido/a: Me esforcé al máximo.

7. Satisfacción con la clase

9 / 10.